Your Car Your Way

Architecture Definition

Chrystopher Deregnaucourt
02/2026

Table des matières

1. Objet du document 2
2. Objectifs du projet 2
3. Principes d'architecture 3
3.1 Principes fondamentaux 3
3.2 Principes de sécurité 4
3.3 Principes de données 4
4. Architecture existante 4
5. Architectures 5
5.1 Architecture Métier 5
5.1.1 Le cas d'utilisation 5
5.1.2 Domaines métier (Domain-Driven Design) 7
5.2 Architecture de Données 9
5.2.1 Diagramme de classes UML 9
5.2.2 Diagramme Entité-Relation 13
5.3 Architecture Technologique 16
5.3.1 Diagramme de composants 16
5.3.2 Diagramme de déploiement 20
6. Justification de l'approche architecturale 23
6.1 Réponse aux objectifs 23
6.2 Respect des principes d'architecture 23
6.3 Conformité aux normes et bonnes pratiques 24
6.4 Analyse des risques et mitigations 24
7. Architectures de transition 25
Annexes 28

1. Objet du document

Ce document permet de modéliser l'architecture qui sera réalisée par les équipes de développement pour le projet **Your Car Your Way**. Au-delà de la modélisation graphique, il s'agit également :

- D'énoncer les principes sur lesquels l'architecture s'appuie
- De justifier cette approche architecturale
- D'indiquer les transitions nécessaires pour la mise en œuvre

Ce document s'appuie sur le **Business Requirements V2** et constitue le cahier des charges technique du projet.

2. Objectifs du projet

Objectifs métier

ID	Objectif	Priorité
OBJ-01	Créer une application web unifiée pour tous les clients internationaux	Haute
OBJ-02	Centraliser la gestion des locations de voitures	Haute
OBJ-03	Améliorer l'expérience client avec un support multicanal (chat, visio)	Haute
OBJ-04	Réduire les avis négatifs grâce à un meilleur service client	Moyenne
OBJ-05	Permettre l'interopérabilité avec les applications internes via API	Haute
OBJ-06	Supporter l'expansion internationale (Europe + USA)	Haute

Objectifs techniques

ID	Objectif	Priorité
TECH-OBJ-01	Garantir une haute disponibilité (99.5%)	Haute
TECH-OBJ-02	Assurer la scalabilité pour 10 000+ utilisateurs simultanés	Haute
TECH-OBJ-03	Respecter les normes de sécurité (RGPD, PCI DSS)	Haute
TECH-OBJ-04	Fournir une API RESTful documentée pour les systèmes internes	Haute
TECH-OBJ-05	Permettre une communication temps réel (WebSocket/WebRTC)	Haute

3. Principes d'architecture

Les principes suivants servent de socle à l'architecture définie :

3.1 Principes fondamentaux

Principe	Description	Justification
Séparation des préoccupations	Chaque composant a une responsabilité unique et bien définie	Facilite la maintenance, les tests et l'évolution
Architecture en couches	Séparation claire entre présentation, logique métier et données	Modularité et indépendance des couches
API-First	L'API est conçue en premier et constitue le contrat d'interface	Permet l'interopérabilité avec les systèmes internes
Microservices modulaires	Services indépendants et faiblement couplés	Scalabilité, déploiement indépendant, résilience
Cloud-Native	Conception pour un déploiement cloud	Élasticité, haute disponibilité, coûts optimisés

3.2 Principes de sécurité

Principe	Description
Defense in Depth	Plusieurs couches de sécurité à chaque niveau
Least Privilege	Accès minimal nécessaire pour chaque composant
Secure by Default	Configuration sécurisée par défaut
Zero Trust	Vérification systématique de toute demande

3.3 Principes de données

Principe	Description
Data Ownership	Chaque service est propriétaire de ses données (ownership logique : schéma dédié + droits DB)
Privacy by Design	Protection des données personnelles dès la conception
Data Consistency	Cohérence éventuelle acceptable pour la performance

4. Architecture existante

Le projet ne s'appuie pas sur un produit existant qui serait repris puis mis à jour. Il n'y a donc pas d'architecture existante à définir.

Contexte : L'entreprise dispose actuellement de plusieurs applications web distinctes par pays. Ces applications ne seront pas migrées mais remplacées par la nouvelle application centralisée.

5. Architectures

5.1 Architecture Métier

5.1.1 Le cas d'utilisation

Description des acteurs et cas d'utilisation

Ce diagramme présente les interactions entre les différents acteurs du système et les fonctionnalités principales de l'application Your Car Your Way.

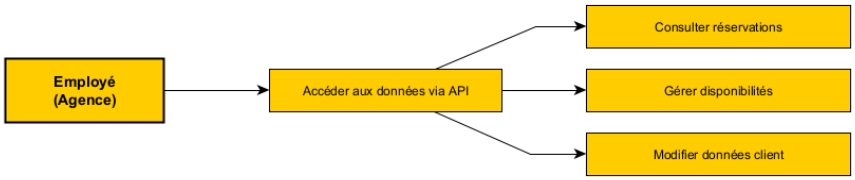
Acteur	Rôle	Cas d'utilisation principaux
Client	Utilisateur final de l'application web. Il peut être un particulier ou un professionnel souhaitant louer un véhicule.	S'inscrire, se connecter, gérer son profil (informations, permis, paiements), rechercher et réserver des véhicules, gérer ses réservations, contacter le support et évaluer ses locations.
Employé (Agence)	Personnel des agences de location utilisant les applications internes. Accède au système via l'API interne.	Consulter les réservations, gérer les disponibilités des véhicules, modifier les données client si nécessaire.
Support Client	Agent du service client répondant aux demandes des utilisateurs via différents canaux.	Répondre aux messages asynchrones, gérer les sessions de chat en direct, participer aux visioconférences avec les clients.

Relations entre cas d'utilisation :

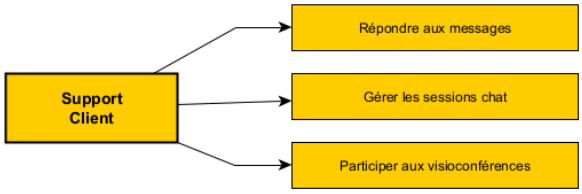
- Les sous-cas d'utilisation (indentés) représentent des fonctionnalités incluses dans le cas parent
- Tous les cas d'utilisation du Client nécessitent une authentification préalable (sauf inscription)
- Le cas "Contacter le support" offre trois canaux : messagerie asynchrone, chat en direct et visioconférence

1) Acteur : Client

2) Acteur : Employé (Agence)



3) Acteur : Support Client



5.1.2 Domaines métier (Domain-Driven Design)

Vue d'ensemble

L'application est découpée en 5 contextes internes (Identity, Booking, Support, Notification, Agency) et 1 contexte **externe** (Payment).
Les contextes **Identity**, **Booking** et **Support** sont reliés au **Payment Context (Externe)** pour gérer les opérations de paiement associées à leurs besoins.

1) Identity Context

Concerne tout ce qui touche à l'identité et au compte utilisateur.

2) Booking Context

Concerne la réservation et l'offre de location.

3) Support Context

Concerne l'assistance et les échanges avec le support client.

4) Payment Context (External)

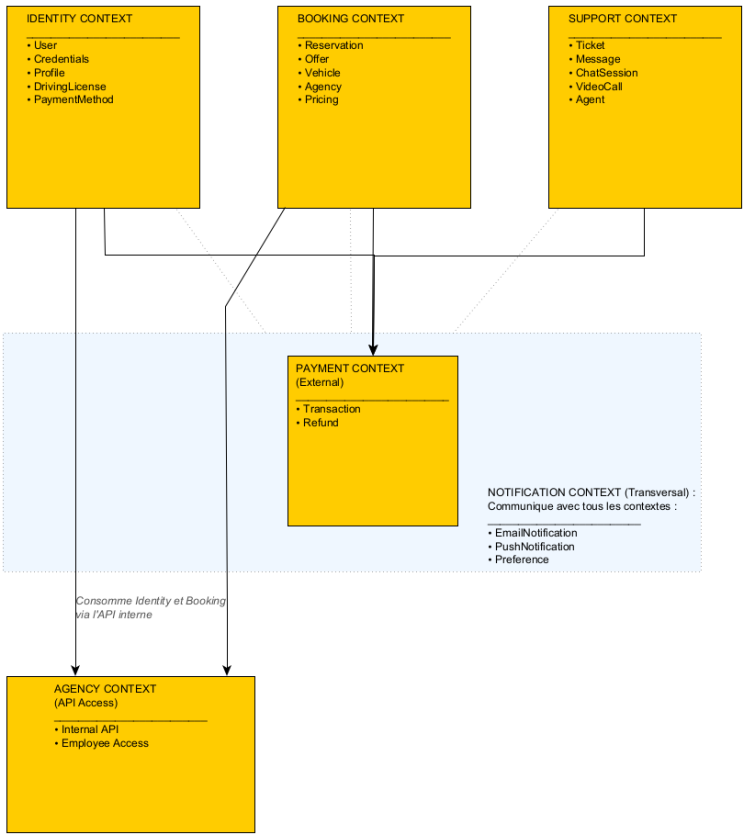
Contexte **externe** au système (service tiers ou module séparé) dédié au paiement.

5) Notification Context

Concerne l'envoi de notifications et les préférences de communication.

6) Agency Context (API Access)

Concerne l'accès "agence/employé" via des API internes.



En résumé : Le système est découpé en contextes métier cohérents, avec un **paiement externalisé** utilisé par plusieurs domaines, et des contextes dédiés à la **notification** et à l'**accès agence via API**.

5.2 Architecture de Données

5.2.1 Diagramme de classes UML

Description des classes du domaine principal.

Le diagramme de classes représente le modèle objet du domaine métier. Pour la lisibilité, le détail du catalogue d'offres (modèles, photos) et du centre d'aide (FAQ/articles) n'est pas entièrement représenté dans le diagramme UML mais est couvert par le schéma de données. Il est divisé en deux parties : le domaine principal (Identity + Booking) et le domaine Support.

Classes du contexte Identity :

Classe	Description	Attributs clés
User	Entité centrale représentant un client. Contient toutes les informations personnelles et préférences.	email, firstName, lastName, country, language, currency, status
DrivingLicense	Permis de conduire du client, requis pour la location. Relation 1:1 avec User.	licenseNumber, issueDate, expiryDate, verified
PaymentMethod	Moyen de paiement enregistré. Un utilisateur peut avoir plusieurs moyens de paiement.	type (card), lastFourDigits, stripePaymentMethodId

Classes du contexte Booking :

Classe	Description	Attributs clés
Reservation	Réservation de véhicule. Entité centrale du processus de location.	confirmationNumber, pickupDateTime, returnDateTime, totalPrice, status
Agency	Agence de location avec sa localisation et ses services.	name, address, latitude/longitude, openingHours, services
VehicleCategory	Catégorie de véhicule selon la norme ACRISS (code 4 caractères).	acrissCode, passengers, luggage, transmission
ReservationOption	Option additionnelle ajoutée à une réservation (GPS, siège enfant…).	optionType, price, quantity

Diagramme de classes - Domaine

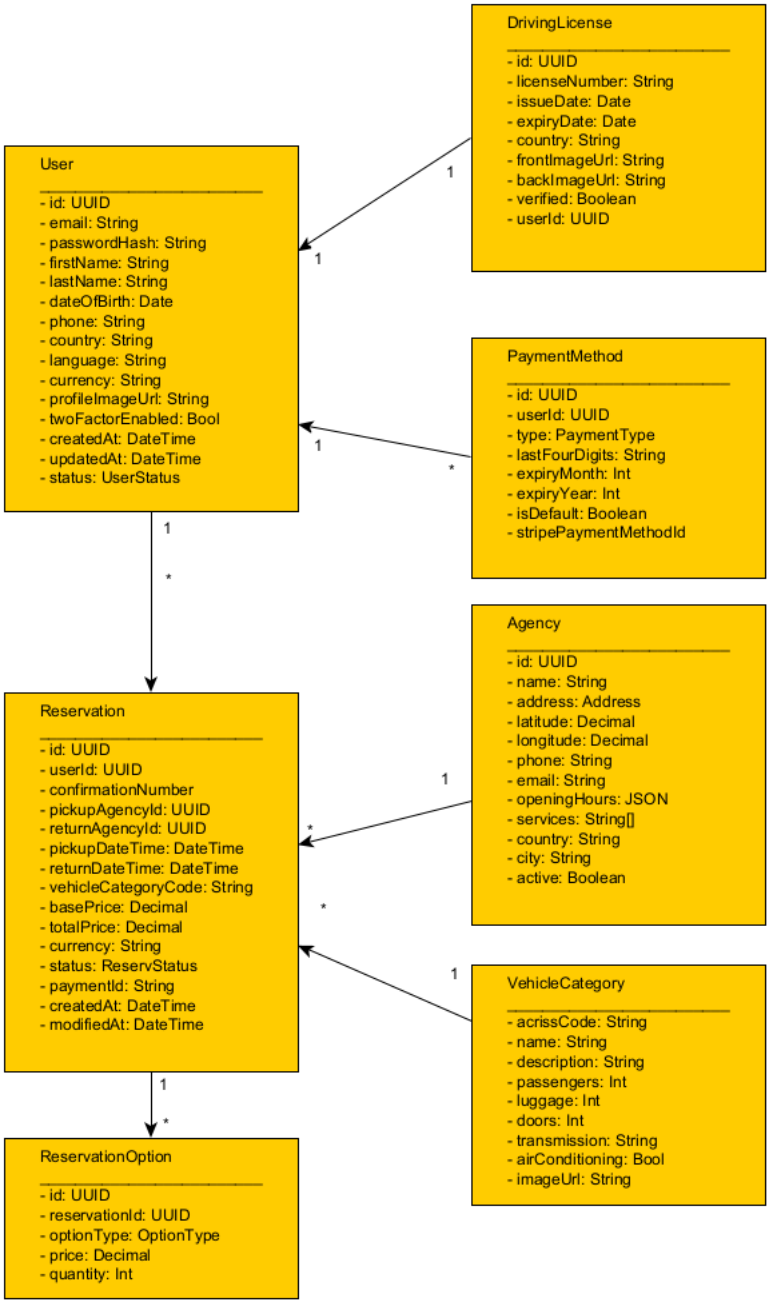
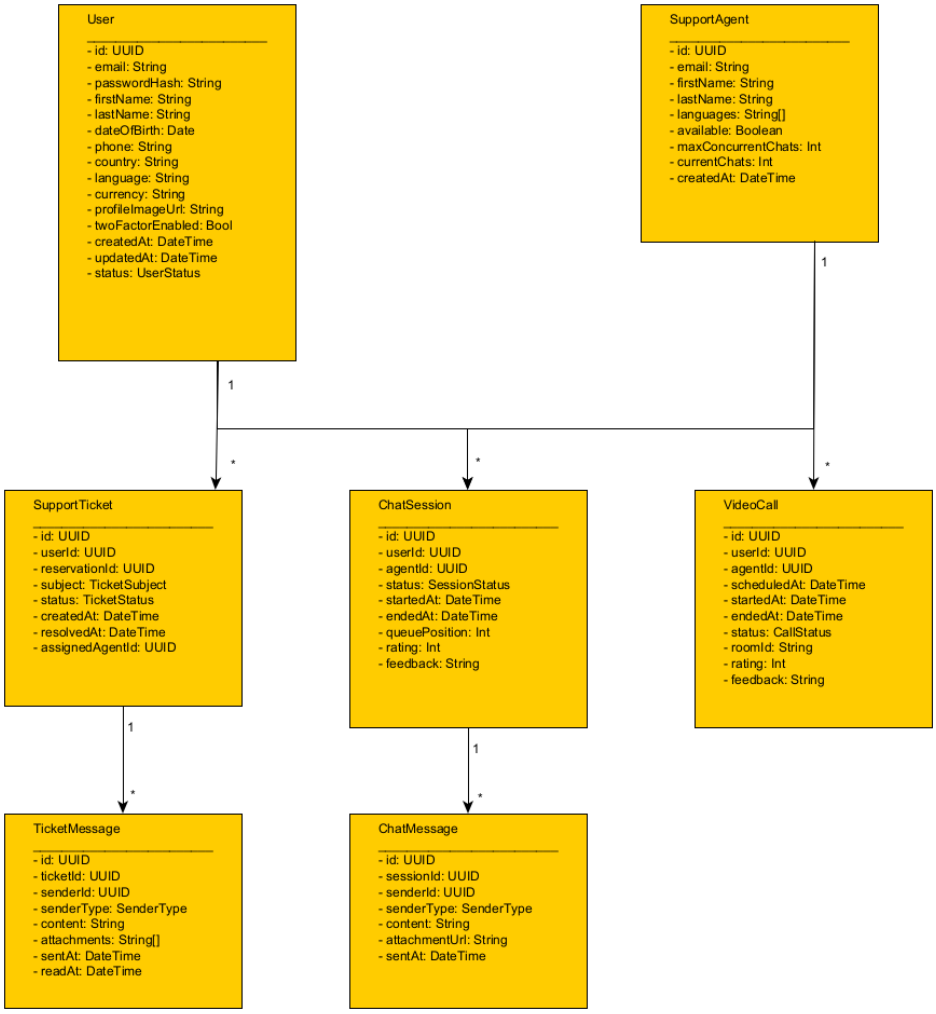


Diagramme de classes – Support

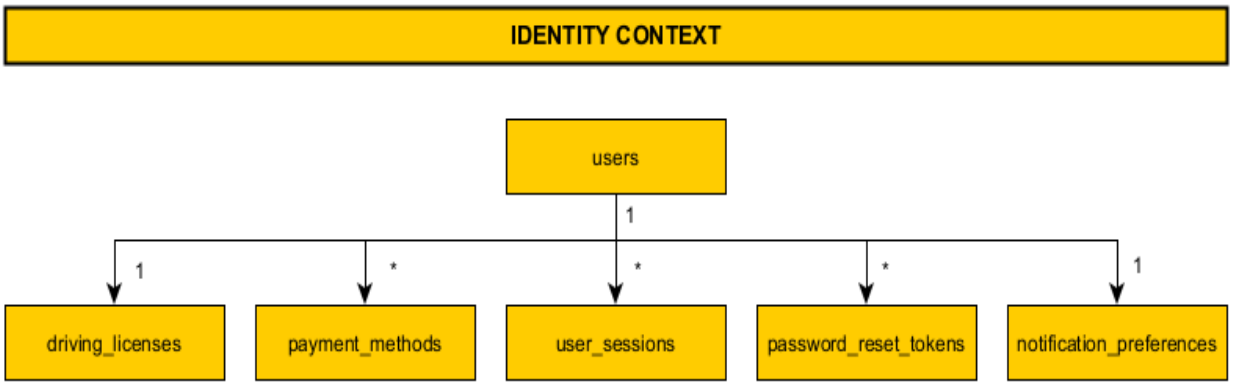


5.2.2 Diagramme Entité-Relation

Description des entités par contexte

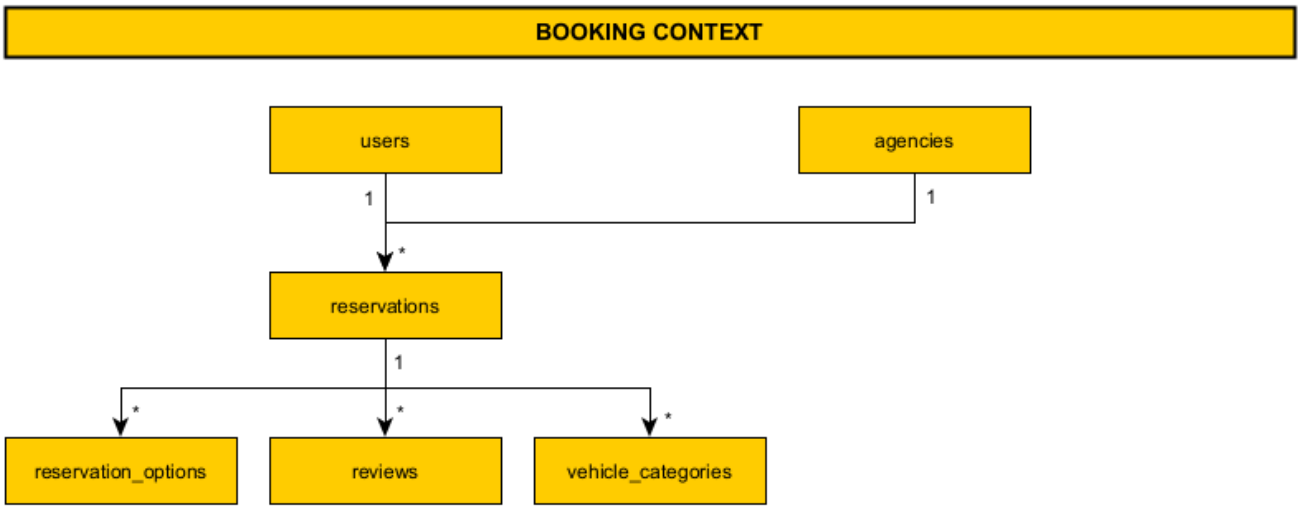
IDENTITY CONTEXT - Gestion de l'identité et de l'authentification des utilisateurs

Entité	Description
users	Table centrale contenant les informations des clients (email, nom, prénom, coordonnées, préférences linguistiques et monétaires). Pivot de tout le système.
driving_licenses	Permis de conduire associé à un utilisateur. Contient le numéro, les dates de validité et les images recto/verso pour vérification.
payment_methods	Moyens de paiement enregistrés (carte bancaire). Stocke uniquement les 4 derniers chiffres et l'ID Stripe pour la sécurité PCI DSS.
user_sessions	Sessions de connexion actives avec token, adresse IP et user agent pour la gestion multi-appareils.
password_reset_tokens	Tokens temporaires pour la réinitialisation de mot de passe avec expiration.
notification_preferences	Préférences de notification par canal (email, push) et par type (booking, support, marketing).



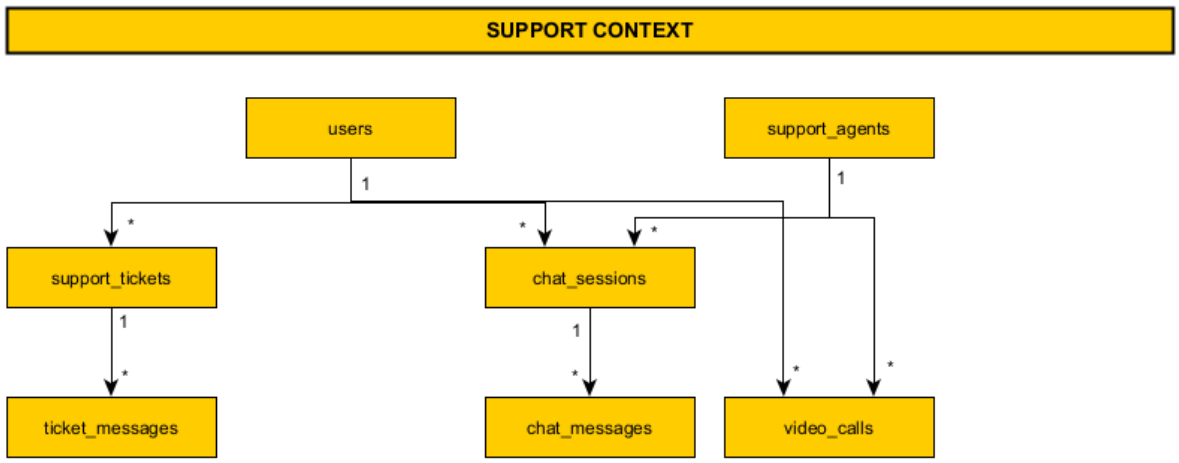
BOOKING CONTEXT - Gestion des réservations de véhicules

Entité	Description
reservations	Réservations de véhicules avec dates de prise en charge/retour, prix, statut et informations de paiement Stripe.
agencies	Agences de location avec adresse, coordonnées GPS, horaires d'ouverture (JSON) et services proposés.
vehicle_categories	Catégories de véhicules selon le code ACRISS (4 caractères). Définit le type, la capacité et les équipements.
vehicle_models	Catalogue de modèles (marque, modèle, caractéristiques) associés à une catégorie ACRISS pour l'affichage « ou similaire ».
vehicle_photos	Photos associées à un modèle ou à une catégorie (stockées sur S3, servies via CloudFront).
reservation_options	Options additionnelles d'une réservation (GPS, siège enfant, conducteur additionnel) avec prix unitaire et quantité.
reviews	Évaluations post-location avec notes (véhicule, service, global) et commentaire.



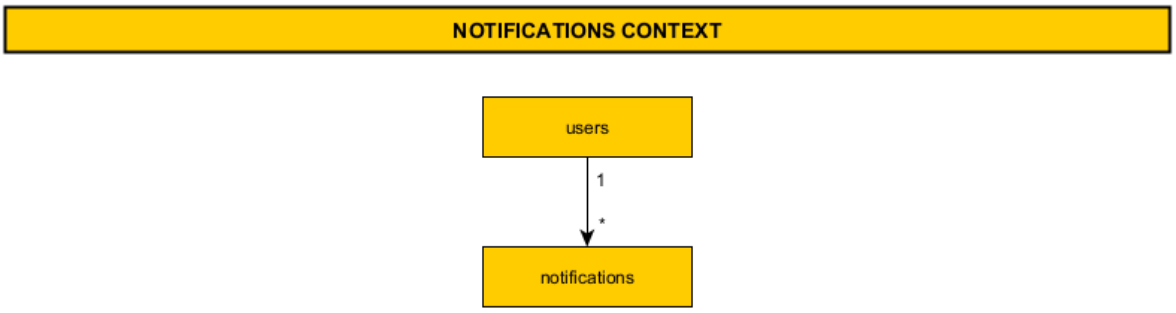
SUPPORT CONTEXT - Gestion du service client multicanal

Entité	Description
support_agents	Agents du service client avec langues parlées, disponibilité et capacité de chats simultanés.
support_tickets	Tickets de support asynchrone avec sujet, statut et agent assigné. Peut être lié à une réservation.
ticket_messages	Messages d'un ticket avec expéditeur (client, agent, système), contenu et pièces jointes.
chat_sessions	Sessions de chat en direct avec statut (attente, actif, terminé), position dans la file et évaluation.
chat_messages	Messages individuels d'une session de chat en temps réel.
video_calls	Appels vidéo planifiés ou en cours avec ID de salle Twilio et évaluation post-appel.
help_categories	Catégories de la base de connaissance (FAQ et guides), utilisées pour l'organisation et le filtrage.
help_articles	Articles d'aide (FAQ, guides) avec contenu, tags et statut de publication.
help_article_translations	Traductions des articles d'aide par langue (i18n), utilisées pour la recherche et l'affichage.



NOTIFICATIONS CONTEXT - Gestion des notifications utilisateur

Entité	Description
notifications	Notifications envoyées aux utilisateurs (email, push, in-app) avec type, titre, contenu et données JSON additionnelles.



5.3 Architecture Technologique

5.3.1 Diagramme de composants

Description de l'architecture en couches

Le diagramme de composants illustre l'architecture logicielle de l'application, organisée en couches distinctes selon le principe de séparation des préoccupations.

Couche Présentation (Presentation Layer)

Composant	Technologie	Responsabilité
Web Application	Angular 17 + TypeScript	Interface utilisateur SPA avec 4 modules principaux
Auth Module	Angular	Inscription, connexion, gestion de session
Booking Module	Angular	Recherche, réservation, paiement
Profile Module	Angular	Gestion du profil, permis, moyens de paiement
Support Module	Angular	Chat, visio, tickets de support, centre d'aide

Couche API Gateway

Composant	Technologie	Responsabilité
API Gateway	Kong / AWS API Gateway	Point d'entrée unique, rate limiting, authentification, SSL termination, load balancing

Couche Logique Métier (Business Logic Layer)

Service	Technologie	Responsabilité
Auth Service	Java 21 + Spring Boot	Gestion des utilisateurs, authentification, 2FA
Booking Service	Java 21 + Spring Boot	Recherche, réservations, agences, tarification
Support Service	Java 21 + Spring Boot	Tickets, gestion des agents, centre d'aide (FAQ, articles)
Payment Service	Java 21 + Spring Boot	Intégration Stripe, webhooks
Notification Service	Java 21 + Spring Boot	Emails (SendGrid), Push (FCM)
Chat Server	Spring WebSocket + STOMP	Communication temps réel, présence
Internal API Service	Java 21 + Spring Boot	API REST pour les applications des agences

Couche Données (Data Layer)

Composant	Technologie	Utilisation
PostgreSQL	RDS	Base de données principale (ACID)
Redis	ElastiCache	Cache, sessions, pub/sub, files d'attente
Elasticsearch	OpenSearch	Recherche full-text (offres + centre d'aide), géolocalisation
S3	AWS S3	Stockage fichiers (images, documents)
CloudFront	AWS CDN	Distribution des assets statiques

Services Externes

Service	Fournisseur	Utilisation
Paiements	Stripe	Transactions, remboursements (PCI DSS)
Emails	SendGrid / AWS SES	Envoi d'emails transactionnels
Vidéo/SMS	Twilio	WebRTC pour visioconférence
OAuth	Google, Apple, Facebook	Connexion sociale

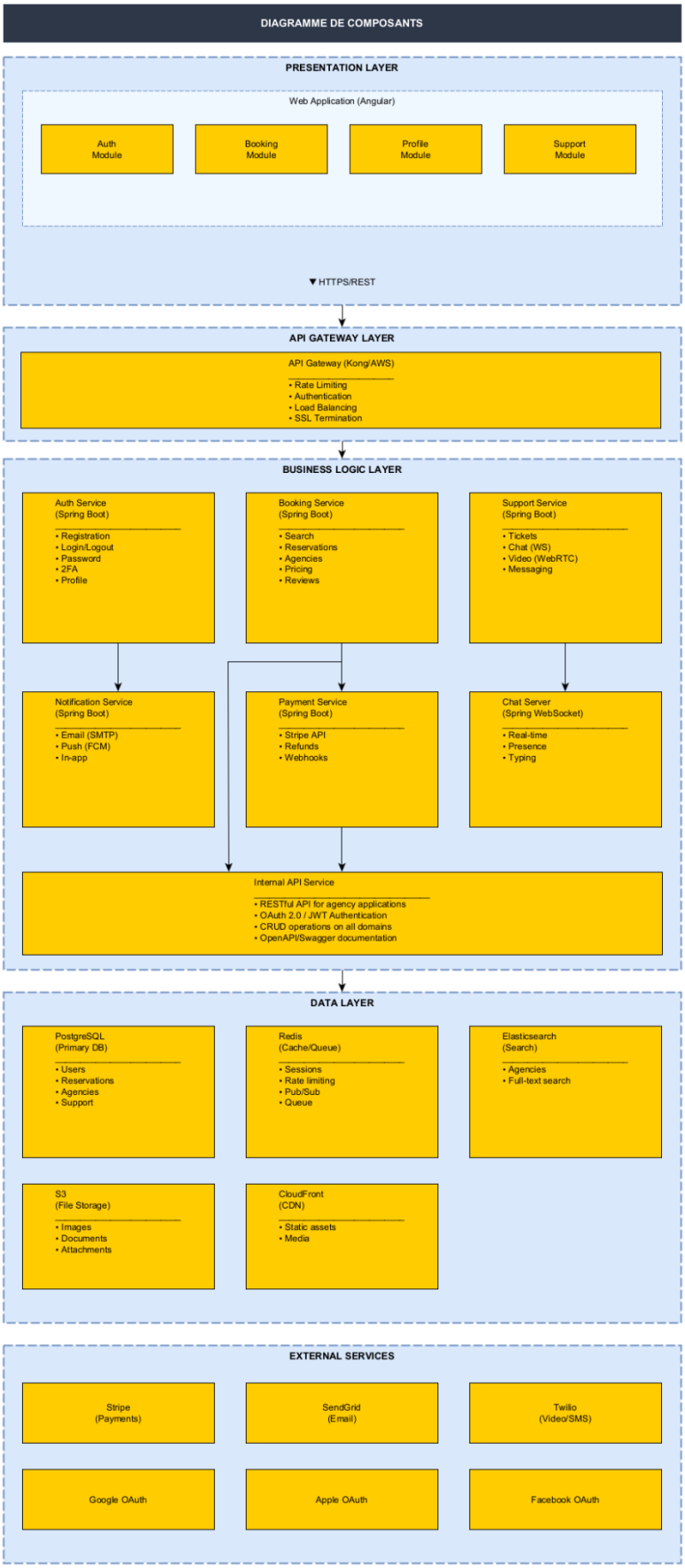


DIAGRAMME DE COMPOSANTS
PRESENTATION LAYER
Web Application (Angular)
Auth Module
Booking Module
Profile Module
Support Module
▼ HTTPS/REST
API GATEWAY LAYER
API Gateway (Kong/AWS)
• Rate Limiting
• Authentication
• Load Balancing
• SSL Termination
BUSINESS LOGIC LAYER
Auth Service (Spring Boot)
• Registration
• Login/Logout
• Password
• 2FA
• Profile
Booking Service (Spring Boot)
• Search
• Reservations
• Agencies
• Pricing
• Reviews
Support Service (Spring Boot)
• Tickets
• Chat (WS)
• Video (WebRTC)
• Messaging
Notification Service (Spring Boot)
• Email (SMTP)
• Push (FCM)
• In-app
Payment Service (Spring Boot)
• Stripe API
• Refunds
• Webhooks
Chat Server (Spring WebSocket)
• Real-time
• Presence
• Typing
Internal API Service
• RESTful API for agency applications
• OAuth 2.0 / JWT Authentication
• CRUD operations on all domains
• OpenAPI/Swagger documentation
DATA LAYER
PostgreSQL (Primary DB)
• Users
• Reservations
• Agencies
• Support
Redis (Cache/Queue)
• Sessions
• Rate limiting
• Pub/Sub
• Queue
Elasticsearch (Search)
• Agencies
• Full-text search
S3 (File Storage)
• Images
• Documents
• Attachments
CloudFront (CDN)
• Static assets
• Media
EXTERNAL SERVICES
Stripe (Payments)
SendGrid (Email)
Twilio (Video/SMS)
Google OAuth
Apple OAuth
Facebook OAuth

5.3.2 Diagramme de déploiement

Description de l'infrastructure AWS

Le diagramme de déploiement présente l'infrastructure cloud hébergée sur AWS, conçue pour la haute disponibilité, la sécurité et la scalabilité.

Composants réseau et sécurité :

Composant	Service AWS	Configuration	Rôle
DNS	Route 53	Zone hébergée	Résolution DNS, health checks
CDN + WAF	CloudFront	Distribution globale	Cache edge, protection DDoS, firewall applicatif
Load Balancer	ALB	Multi-AZ	Répartition de charge, terminaison HTTPS
VPC	VPC	10.0.0.0/16	Isolation réseau, subnets publics/privés
NAT Gateway	NAT	Multi-AZ	Accès internet sortant pour subnets privés

Infrastructure applicative (Private Subnets - APP) :

Service	Instances	Configuration	Scaling
Auth Service	2 tasks Fargate	0.5 vCPU, 1GB RAM	Auto-scaling CPU
Booking Service	3 tasks Fargate	1 vCPU, 2GB RAM	Auto-scaling CPU/mémoire
Support Service	2 tasks Fargate	0.5 vCPU, 1GB RAM	Auto-scaling connexions
Notification Service	2 tasks Fargate	0.25 vCPU, 512MB RAM	Auto-scaling queue
Payment Service	2 tasks Fargate	0.5 vCPU, 1GB RAM	Auto-scaling CPU
Internal API Service	2 tasks Fargate	0.5 vCPU, 1GB RAM	Auto-scaling CPU

Infrastructure données (Private Subnets - DATA) :

Service	Configuration	Haute disponibilité
PostgreSQL	RDS db.r6g.large	Multi-AZ avec réplica standby
Redis	ElastiCache cache.r6g.large	Cluster mode, réplication
Elasticsearch	OpenSearch t3.medium.search	2 nœuds, réplication

Services support :

Service	Utilisation
S3	Stockage des fichiers utilisateur (images permis, pièces jointes)
SES	Envoi d'emails (alternative à SendGrid)
Secrets Manager	Stockage sécurisé des clés API et credentials
CloudWatch	Logs centralisés, métriques, alertes
ECR	Registry des images Docker
CodePipeline	CI/CD automatisé
WAF	Protection contre injections SQL, XSS, bots

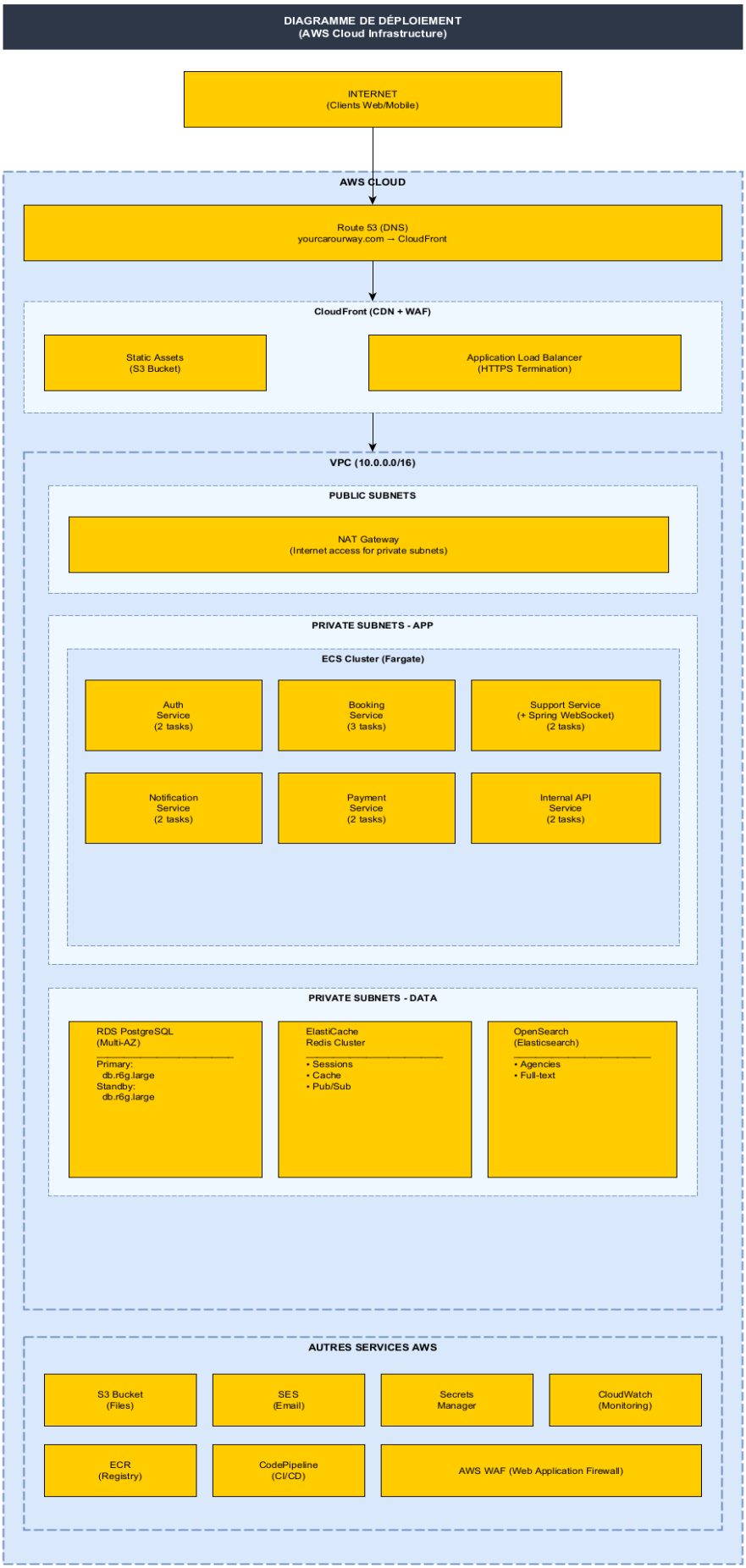


DIAGRAMME DE DÉPLOIEMENT
(AWS Cloud Infrastructure)
INTERNET
(Clients Web/Mobile)
AWS CLOUD
Route 53 (DNS)
yourcarourway.com → CloudFront
CloudFront (CDN + WAF)
Static Assets
(S3 Bucket)
Application Load Balancer
(HTTPS Termination)
VPC (10.0.0.0/16)
PUBLIC SUBNETS
NAT Gateway
(Internet access for private subnets)
PRIVATE SUBNETS - APP
ECS Cluster (Fargate)
Auth
Service
(2 tasks)
Booking
Service
(3 tasks)
Support Service
(+ Spring WebSocket)
(2 tasks)
Notification
Service
(2 tasks)
Payment
Service
(2 tasks)
Internal API
Service
(2 tasks)
PRIVATE SUBNETS - DATA
RDS PostgreSQL
(Multi-AZ)
Primary:
db.r6g.large
Standby:
db.r6g.large
ElastiCache
Redis Cluster
• Sessions
• Cache
• Pub/Sub
OpenSearch
(Elasticsearch)
• Agencies
• Full-text
AUTRES SERVICES AWS
S3 Bucket
(Files)
SES
(Email)
Secrets
Manager
CloudWatch
(Monitoring)
ECR
(Registry)
CodePipeline
(CI/CD)
AWS WAF (Web Application Firewall)

6. Justification de l'approche architecturale

6.1 Réponse aux objectifs

Objectif	Solution architecturale
Application unifiée internationale	Déploiement cloud AWS multi-région, CDN CloudFront pour la distribution globale, i18n intégré
Centralisation des locations	Cluster PostgreSQL managé. Séparation logique par schémas (un par contexte) ; en phase internationale, déploiement par région (UE/USA) pour la résidence des données. API RESTful standardisée
Support multicanal	Spring WebSocket (STOMP) pour le chat temps réel, Twilio pour la visioconférence, architecture événementielle
Interopérabilité API	API RESTful documentée OpenAPI, authentification OAuth 2.0/JWT
Haute disponibilité	ECS Fargate avec auto-scaling, RDS Multi-AZ, load balancing ALB
Scalabilité	Architecture microservices, containers serverless, mise en cache Redis
Sécurité et conformité	WAF, HTTPS, chiffrement at-rest, conformité RGPD/PCI DSS

6.2 Respect des principes d'architecture

Principe	Mise en œuvre
Séparation des préoccupations	Services indépendants (Auth, Booking, Support, etc.), chaque service a sa propre responsabilité
API-First	Spécification OpenAPI rédigée avant le développement, contrat d'interface clair
Cloud-Native	Containers Docker, services managés AWS, infrastructure as code (Terraform)
Defense in Depth	WAF, groupes de sécurité, sous-réseaux privés, chiffrement multicouche

6.3 Conformité aux normes et bonnes pratiques

Norme/Pratique	Conformité
RGPD	Consentement explicite, droit à l'oubli (suppression compte), portabilité des données, DPO désigné
PCI DSS	Paiements délégués à Stripe (Level 1 PCI compliant), pas de stockage de données de carte
OWASP Top 10	Protection XSS, CSRF, injection SQL via ORM, headers de sécurité, rate limiting
REST API	Verbes HTTP appropriés, codes de statut, pagination, HATEOAS
12-Factor App	Configuration par environnement, logs en streams, processus sans état

6.4 Analyse des risques et mitigations

Risque	Probabilité	Impact	Mitigation
Indisponibilité du service	Moyenne	Élevé	Multi-AZ, auto-scaling, health checks
Violation de données	Faible	Critique	Chiffrement, WAF, audits de sécurité
Surcharge du système	Moyenne	Moyen	Rate limiting, cache, auto-scaling
Défaillance du paiement	Faible	Élevé	Fallback Stripe, retry mechanism
Latence internationale	Moyenne	Moyen	CDN, edge locations multiples

7. Architectures de transition

Phase 1 : MVP (Minimum Viable Product) - 3 mois

Périmètre :

- Authentification (inscription, connexion, profil)
- Recherche et réservation de véhicules
- Paiement via Stripe
- Support par messagerie asynchrone

Infrastructure :

- Single region (eu-west-1)
- ECS avec 2 services (Backend + Support)
- RDS PostgreSQL (Single-AZ pour MVP)
- Redis pour sessions

Phase 2 : Fonctionnalités complètes - 2 mois

Ajouts :

- Chat en direct (Spring WebSocket)
- Visioconférence (Twilio)
- Notifications push
- Évaluations et avis

Infrastructure :

- RDS Multi-AZ
- Services dédiés (Auth, Booking, Support, Notification)
- Elasticsearch pour la recherche

Phase 3 : Production internationale - 2 mois

Ajouts :

- API interne pour les agences
- Multi-langue complet
- Conformité RGPD renforcée

Infrastructure :

- Multi-région (eu-west-1 + us-east-1) avec résidence des données par région
- Global Accelerator
- Monitoring avancé (Datadog)

Diagramme de transition

Description des phases de déploiement

La roadmap de transition définit les étapes progressives pour passer du développement initial à la production internationale complète. Chaque phase est conçue pour délivrer de la valeur métier tout en minimisant les risques.

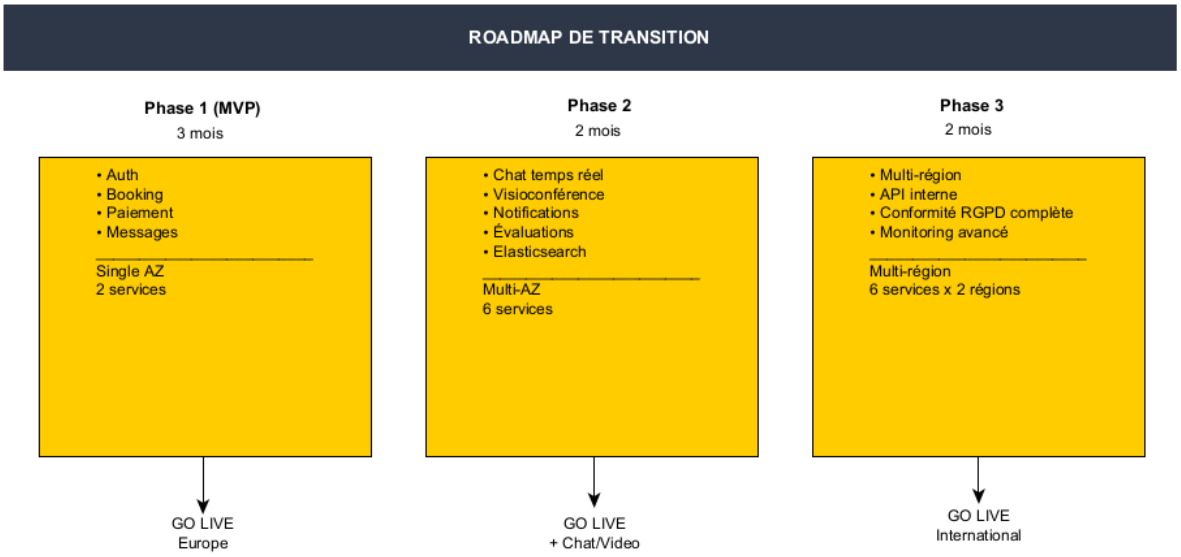
Phase	Durée	Objectif principal	Critères de succès
Phase 1 - MVP	3 mois	Valider le concept avec les fonctionnalités essentielles	1000 utilisateurs actifs, taux de conversion > 5%
Phase 2 - Complet	2 mois	Ajouter les fonctionnalités différenciantes (support temps réel)	NPS > 40, temps de réponse support < 2min
Phase 3 - International	2 mois	Expansion géographique et conformité complète	Disponibilité 99.5%, latence < 200ms monde entier

Évolution de l'infrastructure :

Aspect	Phase 1 (MVP)	Phase 2 (Complet)	Phase 3 (International)
Région AWS	eu-west-1 uniquement	eu-west-1	eu-west-1 + us-east-1
Base de données	Single-AZ	Multi-AZ	Multi-AZ. Données personnelles résidentes par région (UE/USA) ; réplication cross-region limitée aux données non sensibles et à la configuration
Services	2 (Backend monolithique + Support)	6 (microservices)	6 × 2 régions
Recherche	PostgreSQL LIKE	Elasticsearch	Elasticsearch multi-région
Monitoring	CloudWatch basique	CloudWatch + alertes	Datadog complet

Points de validation (Go/No-Go) :

- Fin Phase 1 : Tests de charge réussis, audit sécurité passé
- Fin Phase 2 : Intégration Twilio validée, conformité RGPD vérifiée
- Fin Phase 3 : Tests de latence internationale, certification PCI DSS maintenue


ROADMAP DE TRANSITION
Phase 1 (MVP)
3 mois
• Auth
• Booking
• Paiement
• Messages
Single AZ
2 services
GO LIVE
Europe
Phase 2
2 mois
• Chat temps réel
• Visioconférence
• Notifications
• Évaluations
• Elasticsearch
Multi-AZ
6 services
GO LIVE
+ Chat/Video
Phase 3
2 mois
• Multi-région
• API interne
• Conformité RGPD complète
• Monitoring avancé
Multi-région
6 services x 2 régions
GO LIVE
International

Annexes

A. Technologies alternatives considérées

Composant	Option retenue	Alternatives	Raison du choix
Frontend	Angular	React, Vue.js	Framework complet, architecture enterprise, TypeScript natif
Backend	Java Spring Boot	Node.js, Python Django...	Robustesse enterprise, écosystème mature, typage fort
Database	PostgreSQL	MySQL, MongoDB	JSON support, performance, fiabilité
Cloud	AWS	Azure, GCP	Part de marché, maturité des services
Chat	Spring WebSocket	Socket.IO, Firebase	Intégration native Spring, sécurité intégrée

B. Estimations de coûts AWS (mensuel)

Service	Configuration	Coût estimé
ECS Fargate	6 services × 2 tasks	~370 €
RDS PostgreSQL	db.r6g.large Multi-AZ	~325 €
ElastiCache Redis	cache.r6g.large	~140 €
OpenSearch	t3.medium.search	~93 €
S3 + CloudFront	100GB storage + CDN	~46 €
Autres (ALB, NAT, etc.)	-	~140 €
Total estimé		**~1 114 €/mois**

Document rédigé par : Deregnaucourt Chrystopher
Date : 02/2026
Version : 1.0